

FaaS: Efficient Solution for Data Leakage while Reusing Serverless Computing Environments

람다람쥐: Serverless Platform Research Team

2018147553 류성민

2018147504 윤장한

2018147532 배지호

2018122053 노우준

Abstract

Function-as-a-Service (FaaS) platforms offer scalable computing resources for serverless architectures, yet face challenges in cold-start latency. Container reuse has been proposed as a solution, but raises security concerns regarding data leakage. In this research, we introduce FaaS, a novel architecture that improves FaaS performance through container reuse while addressing security at the application layer. We outline our methodology, including OpenWhisk(open source serverless cloud platform) deployment, security scenario analysis, solution development, validation, and dissemination. Expected outcomes include a more efficient and secure FaaS platform, contributing to advancements in cloud computing.

1. Introduction

Function-as-a-Service (FaaS) is rapidly becoming a preferred cloud computing model due to its ability to offer on-demand, scalable computing resources. FaaS platforms, such as AWS Lambda and Azure Functions, enable developers to deploy individual functions without managing the underlying infrastructure. Despite its advantages, FaaS suffers from cold-start latency, where the initialization of new containers for each function invocation significantly impacts performance. Recent approaches have suggested container reuse as a solution, allowing subsequent invocations to bypass the initialization phase. While this improves performance, it raises security concerns, especially in multi-tenant environments where functions from different users may run on the same physical hardware.

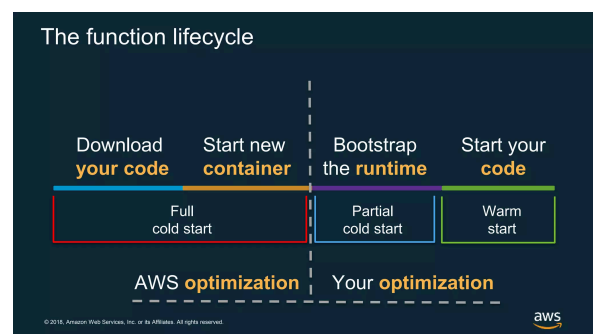


Figure 1. Cold start latency in FaaS [1]

2. Background and Related Work

Existing research on FaaS performance optimization primarily focuses on reducing cold-start latency through various means, including pre-warming of containers [2], [3], and machine learning-based predictive scaling [4]. However, these methods still isolate function executions in separate containers, which cannot fully account for the performance degradation experienced during container initialization. As a solution, recent research suggests successive invocations of functions in the same container [5]. Although this approach lets FaaS avoid cold-start delays, new security issues arise, such as data leakage. In this perspective, memory isolation of each function execution is essential.

3. Proposed Research

We propose a new architecture that leverages container reuse to enhance FaaS performance while introducing an application-layer solution to address the associated security concerns. Our design is centered around an in-memory process snapshot/restore mechanism that efficiently isolates sequential invocations of a function within the same container. This mechanism ensures that any data residue from previous invocations is cleared before a new invocation, thereby preventing data leakage across different users' functions. Unlike existing solutions, our approach is agnostic to the programming language and runtime environment, does not require modifications to existing functions, libraries, or the operating system, and maintains the performance benefits of container reuse.

4. Research Methodology

To initiate our investigation into application-layer security solutions for container reuse in FaaS environments, we will set up a cloud server infrastructure leveraging Apache OpenWhisk [6], an open-source FaaS framework. This setup will serve as our primary testing environment, enabling us to explore and conceptualize FaaS configurations that utilize container reuse to mitigate cold-start latencies while potentially introducing security vulnerabilities.

Our methodology includes the following steps:

4.1 OpenWhisk Deployment: We will deploy the OpenWhisk framework on a cloud server environment. This deployment will allow us to simulate a FaaS platform where we can experiment with container reuse strategies and estimate performance and security.

4.2 Security Scenario Analysis: Utilizing the configured environment, we will execute a series of functions designed to expose potential security vulnerabilities associated with container reuse. These scenarios will be documented to provide a baseline understanding of the specific security challenges we aim to address with our application-layer solution, including the detection and mitigation of potential data leaks that may occur due to inadequate container isolation.

4.3 Solution Development: Building on the insights gained from our security scenario analysis, we will design and implement an application-layer solution tailored to address the identified security concerns in a FaaS context. This solution will aim to isolate function

invocations within reusable containers, ensuring data from previous executions cannot be accessed by subsequent invocations, thereby mitigating the risk of data leaks and unauthorized access.

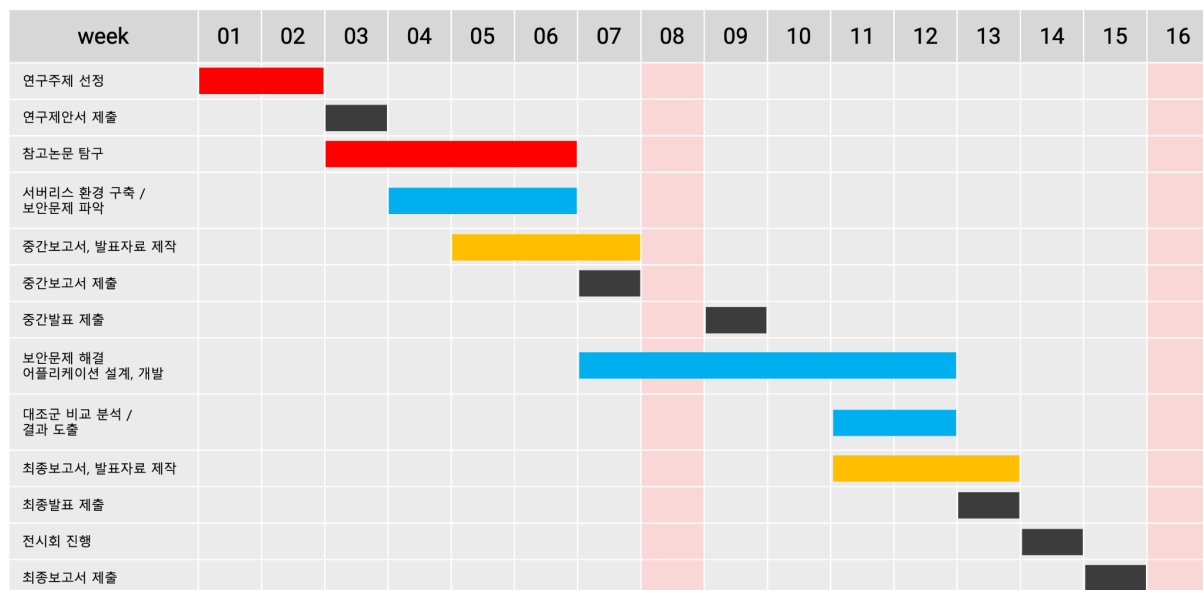
4.4 Validation and Testing: To validate the effectiveness of our proposed solution, we will recreate the security scenarios previously identified and assess whether our application-layer intervention can successfully mitigate the risks. This iterative testing process will allow us to refine our approach, ensuring that it not only enhances security but also preserves or improves the performance benefits of container reuse. By this validation, we aim to demonstrate that enhanced security measures can be implemented without sacrificing operational efficiency. Moreover, our focus on mitigating data leakage issues underscores our commitment to safeguarding sensitive information while maintaining the agility and responsiveness expected in a FaaS environment.

4.5 Documentation: Upon validating our solution, we will document our findings, methodologies, and the specifics of our implementation.

5. Expected Outcomes and Contributions

Our research is expected to contribute to the field of cloud computing, specifically in optimizing FaaS platforms. By addressing the security challenges associated with container reuse, our solution paves the way for more efficient and secure serverless computing. We anticipate demonstrating through extensive evaluation that our architecture can isolate sequential function invocations with minimal overhead, thereby combining the benefits of reduced cold-start latency with enhanced security.

6. Research schedule



References

- [1] “Become a Serverless Black Belt - Optimizing Your Serverless Applications - AWS Online Tech Talks,” SlideShare. Accessed: Mar. 19, 2024. [Online]. Available: <https://www.slideshare.net/AmazonWebServices/become-a-serverless-black-belt-optimizing-your-serverless-applications-aws-online-tech-talks>
- [2] D. Du *et al.*, “Catalyzer: Sub-millisecond Startup for Serverless Computing with Initialization-less Booting,” in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, in ASPLOS ’20. New York, NY, USA: Association for Computing Machinery, 13 2020, pp. 467–481. doi: 10.1145/3373376.3378512.
- [3] D. Ustiugov, P. Petrov, M. Kogias, E. Bugnion, and B. Grot, “Benchmarking, Analysis, and Optimization of Serverless Function Snapshots,” in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Apr. 2021, pp. 559–572. doi: 10.1145/3445814.3446714.
- [4] Z. Zhou, Y. Zhang, and C. Delimitrou, “AQUATOPE: QoS-and-Uncertainty-Aware Resource Management for Multi-stage Serverless Workflows,” in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, in ASPLOS 2023. New York, NY, USA: Association for Computing Machinery, 21 2022, pp. 1–14. doi: 10.1145/3567955.3567960.
- [5] M. Alzayat, J. Mace, P. Druschel, and D. Garg, “Groundhog: Efficient Request Isolation in FaaS,” arXiv.org. Accessed: Mar. 19, 2024. [Online]. Available: <https://arxiv.org/abs/2205.11458v1>
- [6] “Apache OpenWhisk is a serverless, open source cloud platform.” Accessed: Mar. 19, 2024. [Online]. Available: <https://openwhisk.apache.org/>